

Project: Human Resource Management System (HRMS)



Session: 2025 – 2029

Submitted by:

Uswa Nawaz 2025-CS-312

Supervised by:

Prof. Nimra Akram

Department of Computer Science

University of Engineering and Technology Lahore Pakistan

Contents

Introduction	5
Project Overview	5
Problem Statement.....	5
Objective / Solution	5
Users	6
Admin	6
Employee.....	6
Features	6
Role-Based Authentication.....	6
First-Time System Initialization.....	7
Admin Dashboard.....	7
Employee Management.....	7
Department Management	7
Payroll and Salary Slips	7
Leave Management.....	7
Work Schedule Management.....	7
Promote and Demote	7
Employee Dashboard.....	8
Password Management	8
Data Validation	8
Wireframes	8
Login Form.....	8
Initial Setup Form.....	9
Admin Dashboard - Employees Tab	9
Admin Dashboard - Departments Tab	10
Admin Dashboard - Payroll Tab	11
Admin Dashboard - Leaves Tab	11
Admin Dashboard - Schedules Tab	12
Add Employee Form.....	12
Promote / Demote Form.....	13

Human Resource Management System

Update Employee Form	13
Salary Slip Form.....	14
Employee Dashboard - My Profile Tab	14
Employee Dashboard - Salary Tab	15
Employee Dashboard - Leaves Tab	15
Employee Dashboard - Security Tab	16
CRC Diagram.....	16
OOP Concepts Used in Each Class	18
Interfaces	18
ISaveable.cs	18
ISearchable.cs	18
IValidatable.cs	18
Utility Classes	18
Validation.cs	18
Business Logic Layer (BL)	19
PersonBL.cs.....	19
UserBL.cs.....	19
EmployeeBL.cs	20
AdminBL.cs	21
DepartmentBL.cs	21
SalarySlipBL.cs	22
LeaveBL.cs.....	22
Data Layer (DL)	23
UserDL.cs.....	23
EmployeeDL.cs	23
DepartmentDL.cs.....	23
LeaveDL.cs	24
SalaryDL.cs	24
ScheduleDL.cs.....	24
Forms / UI Layer	25
LoginForm.cs	25
InnitSetupForm.cs.....	25

Human Resource Management System

AdminDashboard.cs.....	25
EmployeeDashboard.cs.....	26
addEmployeeForm.cs.....	26
UpdateEmployeeForm.cs.....	26
PromoteDemoteForm.cs	26
SalarySlipForm.cs.....	27
Main Driver Program.....	27
Program.cs.....	27

Introduction

Project Overview

The Human Resource Management System (HRMS) is a desktop application built using C# with Windows Forms and SQL Server. It is designed to help organizations manage their employees and HR-related tasks from a single system.

The system has two types of users. The first is the admin, who is responsible for adding and managing employees, handling payroll, approving leaves, and assigning work schedules. The second is the Employee, who can log in to view their own profile, check their salary slips, submit leave requests, and update their password.

The application uses a role-based login system. This means it automatically detects whether the person logging in is an Admin or an Employee and opens the correct dashboard for them.

Problem Statement

- **No central record keeping**
Employee information like salary, department, and contact details was stored in scattered files or spreadsheets, making it hard to find or update.
- **Manual salary calculations**
Payroll was calculated by hand, which was slow and often resulted in errors, especially when calculating tax deductions.
- **Inefficient Leave Tracking:**
Leave requests were made verbally or through messages with no formal way to track whether a request was approved, rejected, or still pending.
- **Data Security Risks:**
There was no way to control who could access HR data, meaning sensitive employee information was not protected.
- **Informal Shift Scheduling:**
Work schedules were communicated informally, making it difficult to keep a proper record of who was assigned to which shift.
- **Scalability Challenges:**
As the number of employees increases, managing everything manually becomes increasingly time-consuming and error-prone.

Objective / Solution

- All employee records are stored in a SQL Server database and can be added, updated, searched, and deleted easily from one place.
- Salary slips are generated automatically with gross salary, 7% tax deduction, and net salary calculated by the system without any manual effort.
- Employees can submit leave requests through the app and track their status. Admins can approve or reject requests from a single screen.

Human Resource Management System

- A login system ensures that only authorized users can access the system, and each user only sees what their role allows.
- Admins can assign shifts to employees from predefined options, and employees can view their assigned schedule from their own dashboard.
- The system is built using OOP principles with a clean layered structure, making it easy to maintain and expand in the future.

Users

Admin

The admin is the system manager, typically an HR officer or manager. The admin has full control over the system and is responsible for managing all HR operations.

The admin can:

- Add, view, search, update, and delete employee records
- Add and delete departments
- Generate salary slips for employees
- Approve or reject employee leave requests
- Assign work shifts to employees
- Promote or demote employees by changing their designation

Employee

The Employee is a regular staff member who has limited access to the system. Employees can only view and manage their own personal information.

The employee can:

- View their own profile including name, department, designation, salary, and assigned shift
- View their salary slips
- Submit leave requests and track their approval status
- Change their own password

Features

Role-Based Authentication

The login screen is the entry point of the application. The user enters their username and password and the system automatically identifies whether they are an Admin or an Employee. Based on the role, the correct dashboard is opened. If the credentials do not match any record, an error message is shown.

First-Time System Initialization

When the application is launched for the very first time and no admin account exist in the database, the system automatically opens a setup screen. This screen allows the creation of the first master admin account. After setup is complete, the system proceeds to the normal login screen. This screen never appears again once an admin exists.

Admin Dashboard

The admin dashboard is a single window with five tabs, each covering a different area of HR management. This keeps everything organized in one place without opening multiple windows.

Employee Management

The admin can add new employees by filling in their details including name, username, password, email, phone, salary, and department. The system automatically generates a unique Employee ID. The admin can also view all employees in a table, search by name, department, or ID, update existing records, delete employees, and promote or demote them using a predefined list of designations.

Department Management

The admin can add new departments and delete existing ones. If a department still has employees assigned to it, the system prevents deletion and shows a warning message. Departments are displayed in alphabetical order.

Payroll and Salary Slips

The admin selects an employee ID and a month from a dropdown and generates a salary slip. The system automatically calculates the 7% tax deduction and net salary. The slip is displayed in a clean formatted window showing gross salary, tax amount, and net salary. A slip cannot be generated twice for the same employee and month.

Leave Management

Employees can submit leave requests from their dashboard by entering a reason and selecting from and to dates. All submitted requests appear in the admin's Leaves tab where the admin can approve or reject them. Once a request has been processed it cannot be changed again. Employees can track the status of their requests from their own dashboard.

Work Schedule Management

The admin can assign a work shift to any employee by entering their employee ID and selecting a shift name, start time, and end time from dropdowns. The available shifts are Morning, Evening, and Night. If an employee already has a schedule assigned, it is replaced with the new one. Employees can view their assigned shift from their profile tab.

Promote and Demote

The admin can change the designation of any employee through a dedicated form. The new designation is selected from a predefined dropdown list containing options such as Intern, Associate, Team Lead, Manager, Director, and CEO. Free text entry is not allowed to prevent invalid data.

Employee Dashboard

After logging in, the employee sees a personal dashboard with four tabs. The profile tab shows their full information including name, department, designation, salary, email, phone, and assigned shift. The salary tab shows all their generated salary slips. The leaves tab allows them to submit requests and track statuses. The security tab allows them to change their password.

Password Management

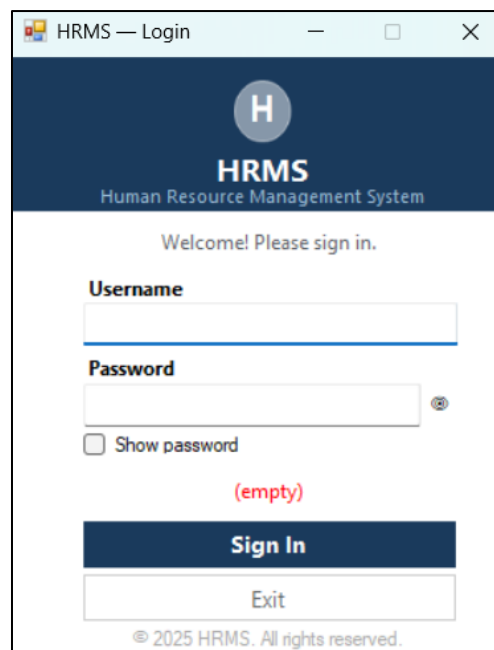
The employees can change their password from their respective dashboards. The system verifies the current password before allowing a change. The new password must meet the format rules which require at least one capital letter, one number, and one special character. A show and hide password button is on all the password fields throughout the system.

Data Validation

All input in the system is validated before being saved to the database. Validation is handled by a dedicated static Validations class in the library. Rules include name must start with a capital letter and contain no numbers, username must be between 4 and 20 characters, password must be 8 to 15 characters with at least one capital letter, one number, and one special character, phone must be exactly 11 digits, and salary must be between 40,000 and 1,000,000. Error messages are shown in red directly on the form without any popups.

Wireframes

Login Form



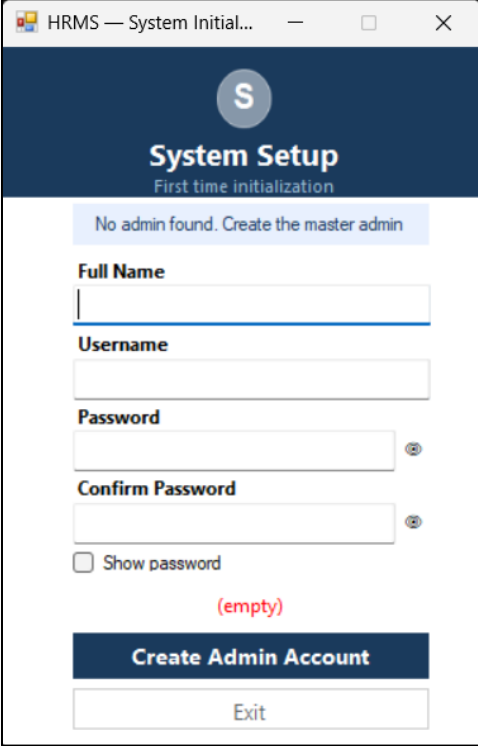
The wireframe shows a login window titled "HRMS — Login". The header features a dark blue bar with a white "H" in a circle, the text "HRMS", and "Human Resource Management System" below it. The main content area is white and contains the text "Welcome! Please sign in." followed by two input fields: "Username" and "Password". The "Password" field has a "Show password" checkbox and a red "(empty)" message below it. At the bottom are two buttons: "Sign In" (dark blue) and "Exit" (white). The footer text is "© 2025 HRMS. All rights reserved."

Human Resource Management System

This is the main entry point of the application. The user enters their username and password to access the system. The system automatically identifies the role and redirects to the correct dashboard. If no admin account exists, the system skips to the Initial Setup screen instead.

Initial Setup Form

This screen appears only once when the application is launched for the very first time and no admin account is found in the database. It allows the creation of the master administrator account. Once completed, this screen never appears again.



The screenshot shows a web browser window titled "HRMS — System Initial...". The page has a dark blue header with a white "S" in a circle, the text "System Setup", and "First time initialization" below it. A light blue message box states "No admin found. Create the master admin". Below this are four input fields: "Full Name", "Username", "Password", and "Confirm Password". The "Password" and "Confirm Password" fields have eye icons for toggling visibility. A "Show password" checkbox is located below the "Confirm Password" field. A red "(empty)" label is positioned above the "Create Admin Account" button. At the bottom are two buttons: "Create Admin Account" (dark blue) and "Exit" (light gray).

Admin Dashboard - Employees Tab

The Employees tab allows the admin to view all registered employees in a table. The admin can add a new employee, search by name, department, or employee ID, update an existing record, delete an employee, or change their designation using the action buttons.

Human Resource Management System

The screenshot shows the HRMS Admin Dashboard with the 'Employees' tab selected. The dashboard includes a header with the user 'uswa' and a 'Logout' button. Below the header are tabs for 'Employees', 'Departments', 'Payroll', 'Leaves', and 'Schedules'. A search bar with a 'Search' button and a '+ Add Employee' button are present. The main area displays a table of employees with columns: EmpID, Name, DeptName, Designation, Salary, Email, Phone, and Username. The table contains five rows of employee data. At the bottom, there are buttons for 'Update', 'Delete Selected', and 'Promote / Demote'.

EmpID	Name	DeptName	Designation	Salary	Email	Phone	Username
EMP-1004	Amna Ijaz	Marketing	Associate	90000.00	amna2@gm...	01254789...	amna_2
EMP-1002	Ayan	Marketing	Manager	60000.00	ali8cs@gm...	03247876...	ali01
EMP-1001	Azeem	Finance	Senior Ass...	50000.00	azeemwor...	03189652...	azeem_11
EMP-1003	Fizza Ali	HR	Manager	10000.00	fizza7@gm...	03157489...	fizza_09
EMP-1005	Zainab Asif	Maintaina...	Director	150000.00	zainab66@...	03147856...	zainab

Admin Dashboard - Departments Tab

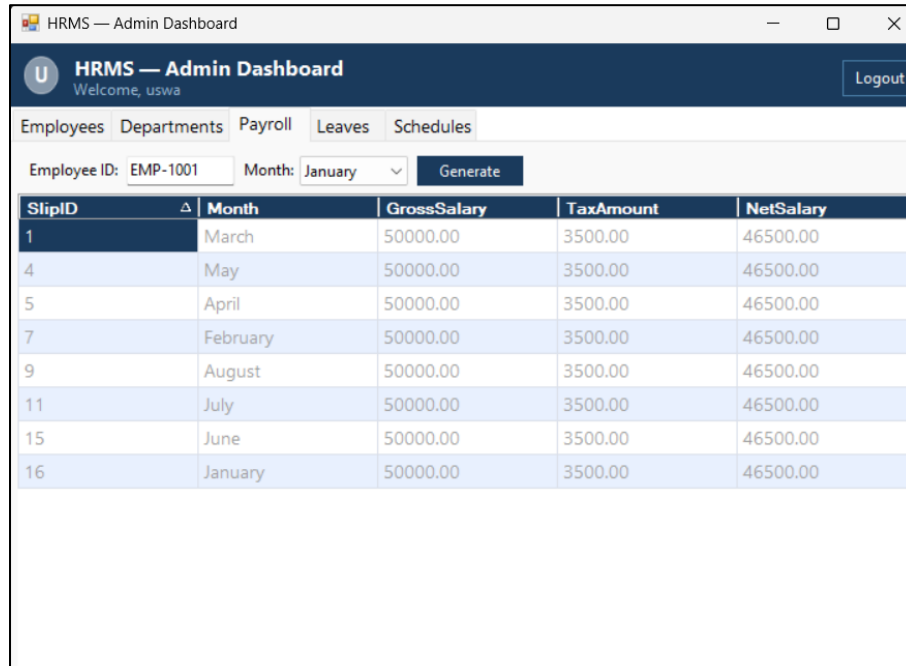
The Departments tab displays all existing departments. The admin can add a new department by typing its name and clicking Add, or delete a selected department. Departments that still have employees assigned cannot be deleted.

The screenshot shows the HRMS Admin Dashboard with the 'Departments' tab selected. The dashboard includes a header with the user 'uswa' and a 'Logout' button. Below the header are tabs for 'Employees', 'Departments', 'Payroll', 'Leaves', and 'Schedules'. A form for adding a department is visible, with a 'Department Name' input field, a '+ Add' button, and a 'Delete' button. The main area displays a table of departments with columns: DeptID and DeptName. The table contains five rows of department data. At the bottom, there are buttons for 'Update', 'Delete Selected', and 'Promote / Demote'.

DeptID	DeptName
4	Finance
2	HR
1	IT
8	Maintainance
7	Marketing

Admin Dashboard - Payroll Tab

The payroll tab is used to generate salary slips for employees. The admin enters an employee ID and selects a month from the dropdown. The system retrieves the salary, calculates the 7% tax deduction, and generates a formatted salary slip.

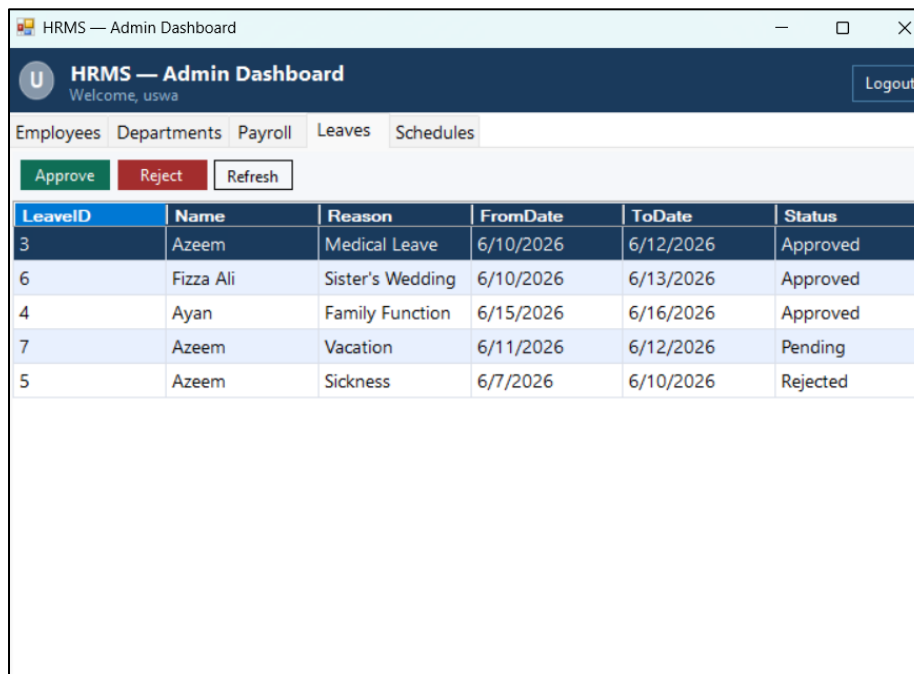


The screenshot shows the HRMS Admin Dashboard with the Payroll tab selected. The interface includes a header with the HRMS logo, a user profile icon, and a 'Logout' button. Below the header, there are tabs for 'Employees', 'Departments', 'Payroll', 'Leaves', and 'Schedules'. The 'Payroll' tab is active, displaying a form with 'Employee ID: EMP-1001' and a 'Month' dropdown set to 'January'. A 'Generate' button is next to the form. Below the form is a table with the following data:

SlipID	Month	GrossSalary	TaxAmount	NetSalary
1	March	50000.00	3500.00	46500.00
4	May	50000.00	3500.00	46500.00
5	April	50000.00	3500.00	46500.00
7	February	50000.00	3500.00	46500.00
9	August	50000.00	3500.00	46500.00
11	July	50000.00	3500.00	46500.00
15	June	50000.00	3500.00	46500.00
16	January	50000.00	3500.00	46500.00

Admin Dashboard - Leaves Tab

The Leaves tab shows all leave requests submitted by employees across the organization. The admin can select a pending request and either approve or reject it. Once a decision is made it cannot be changed again.



The screenshot shows the HRMS Admin Dashboard with the Leaves tab selected. The interface includes a header with the HRMS logo, a user profile icon, and a 'Logout' button. Below the header, there are tabs for 'Employees', 'Departments', 'Payroll', 'Leaves', and 'Schedules'. The 'Leaves' tab is active, displaying a table with leave requests. Above the table are buttons for 'Approve', 'Reject', and 'Refresh'.

LeaveID	Name	Reason	FromDate	ToDate	Status
3	Azeem	Medical Leave	6/10/2026	6/12/2026	Approved
6	Fizza Ali	Sister's Wedding	6/10/2026	6/13/2026	Approved
4	Ayan	Family Function	6/15/2026	6/16/2026	Approved
7	Azeem	Vacation	6/11/2026	6/12/2026	Pending
5	Azeem	Sickness	6/7/2026	6/10/2026	Rejected

Admin Dashboard - Schedules Tab

The Schedules tab allows the admin to assign a work shift to any employee. The admin enters the employee ID and selects the shift name, start time, and end time from the dropdowns. If a schedule already exists for that employee, it is automatically replaced.

The screenshot shows the 'Schedules' tab in the HRMS Admin Dashboard. At the top, there's a navigation bar with 'Employees', 'Departments', 'Payroll', 'Leaves', and 'Schedules'. Below this, there's a form to assign a shift: 'Emp ID:' (text input), 'Shift' (dropdown menu showing 'Morning Shift'), 'Start' (dropdown menu showing '08:00 AM'), 'End:' (dropdown menu showing '04:00 PM'), and an 'Assign' button. Below the form is a table of employee schedules.

EmpID	Name	ShiftName	StartTime	EndTime
EMP-1004	Amna Ijaz	Night Shift	03:00 PM	11:00 PM
EMP-1002	Ayan	Evening Shift	04:00 PM	11:00 PM
EMP-1001	Azeem	Evening Shift	08:00 AM	12:00 AM
EMP-1003	Fizza Ali	Evening Shift	01:00 PM	10:00 PM
EMP-1005	Zainab Asif	Morning Shift	08:00 AM	04:00 PM

Add Employee Form

This form opens when the admin clicks the Add Employee button. The admin fills in the employee's full name, username, password, email, phone number, salary, and selects a department from the dropdown. The system validates all fields and automatically generates a unique employee ID before saving.

The screenshot shows the 'Add New Employee' form overlaying the 'Employees' tab in the HRMS Admin Dashboard. The form has fields for 'Full Name:', 'Username:', 'Password:', 'Email:', 'Phone:', 'Salary:', and 'Department:' (dropdown menu showing 'Finance'). There are 'Save Employee' and 'Cancel' buttons at the bottom. In the background, a table of employees is visible.

EmpID	Name	Phone	Username
EMP-1004	Amna Ijaz	01254789...	amna_2
EMP-1002	Ayan	03247876...	ali01
EMP-1001	Azeem	03189652...	azeem_11
EMP-1003	Fizza Ali	03157489...	fizza_09
EMP-1005	Zainab Asif	03147856...	zainab

Promote / Demote Form

This form opens when the admin selects an employee and clicks Promote/Demote. It shows the employee's current designation and provides a dropdown of predefined roles to choose from. Free text entry is not allowed to prevent invalid designations from being saved.

The screenshot shows the HRMS Admin Dashboard with a modal window titled "Promote / Demote Employee". The modal displays the employee's current designation as "Associate" and a dropdown menu for selecting a new designation. The background shows a table of employees with columns: EmpID, Name, Department, Phone, and Username.

EmpID	Name	Department	Phone	Username
EMP-1004	Amna Ijaz	Marketing	01254789...	amna_2
EMP-1002	Ayan	Marketing	03247876...	ali01
EMP-1001	Azeem	Finance	03189652...	azeem_11
EMP-1003	Fizza Ali	HR	03157489...	fizza_09
EMP-1005	Zainab Asif	Marketing	03147856...	zainab

Update Employee Form

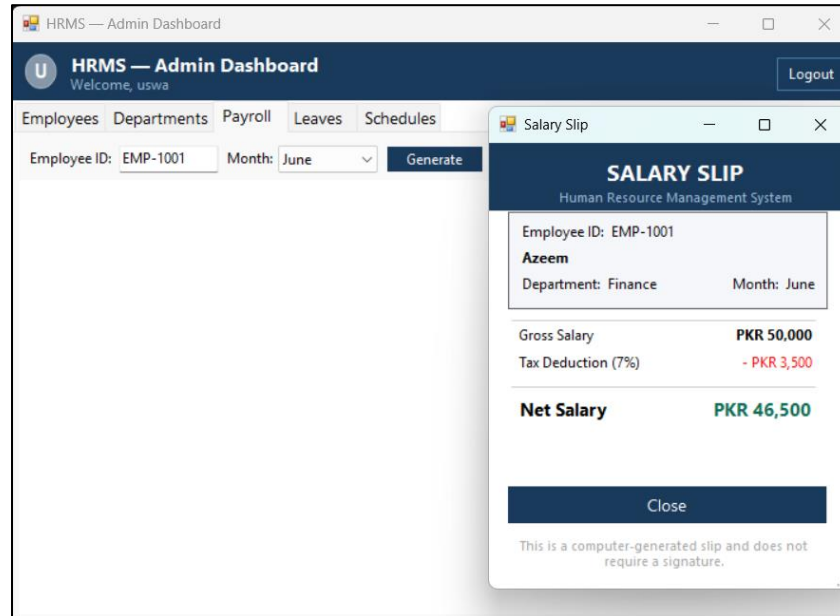
This form opens when the admin click update button, allows the admin to edit name, email, phone number, salary and department of the employee.

The screenshot shows the HRMS Admin Dashboard with a modal window titled "Update Employee Details". The modal displays fields for Full Name, Email, Phone, Salary, and Department. The background shows a table of employees with columns: EmpID, Name, Department, Phone, and Username.

EmpID	Name	Department	Phone	Username
EMP-1004	Amna Ijaz	Marketing	01254789...	amna_2
EMP-1002	Ayan	Marketing	03247876...	ali01
EMP-1001	Azeem	Finance	03189652...	azeem_11
EMP-1003	Fizza Ali	HR	03157489...	fizza_09
EMP-1005	Zainab Asif	Marketing	03147856...	zainab

Salary Slip Form

This window displays the generated salary slip for an employee. It shows the employee ID, name, department, and selected month at the top, followed by the gross salary, tax deduction amount, and final net salary. This form is also accessible to employees from their own dashboard.

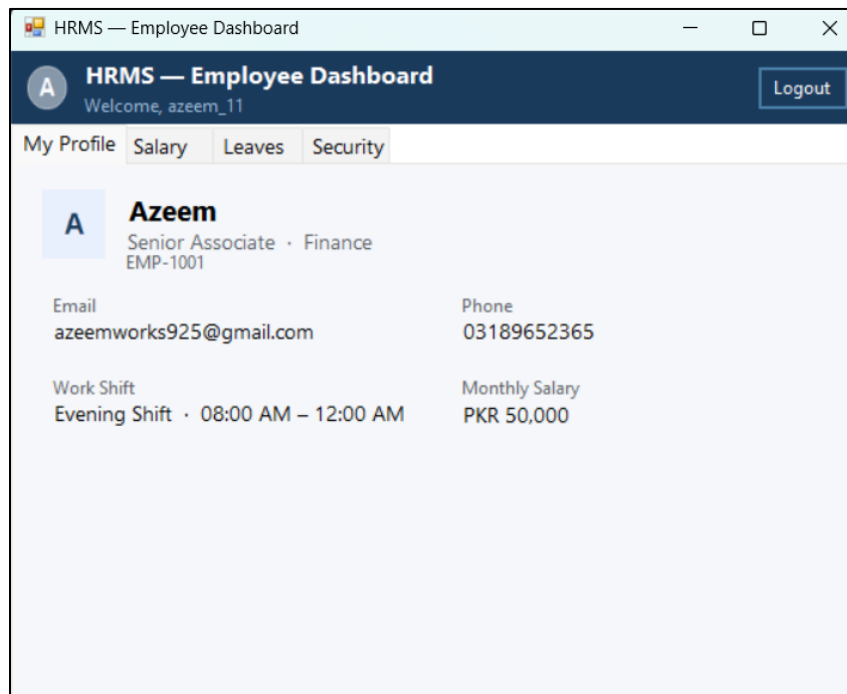


The screenshot shows the HRMS Admin Dashboard with a 'Salary Slip' modal window open. The modal displays the following information:

SALARY SLIP	
Human Resource Management System	
Employee ID: EMP-1001	
Azeem	
Department: Finance	Month: June
Gross Salary	PKR 50,000
Tax Deduction (7%)	- PKR 3,500
Net Salary	PKR 46,500
Close	
This is a computer-generated slip and does not require a signature.	

Employee Dashboard - My Profile Tab

This is the first screen an employee sees after logging in. It displays their personal information including name, designation, department, email, phone number, monthly salary, and assigned work shift. This tab is read-only and cannot be edited by the employee.

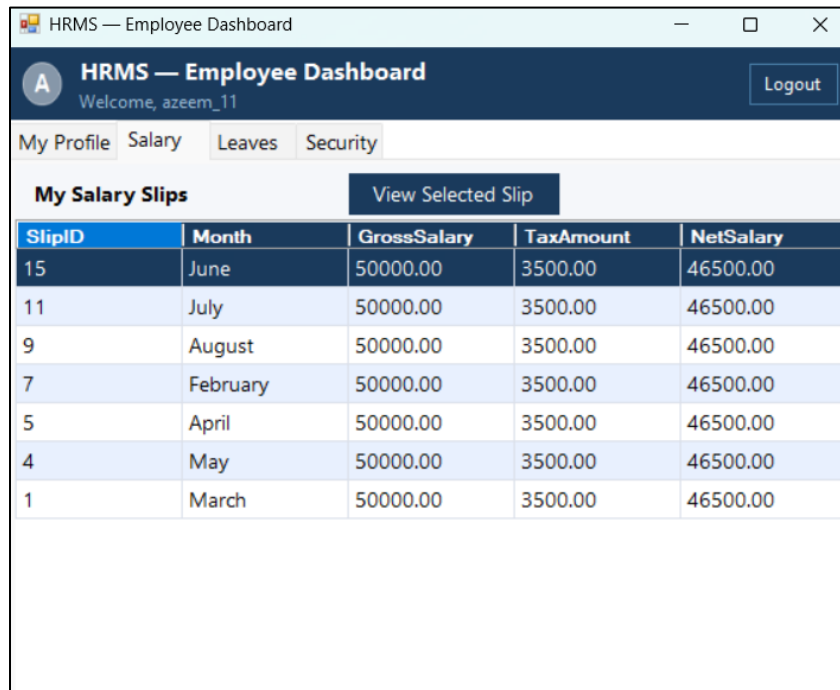


The screenshot shows the HRMS Employee Dashboard with the 'My Profile' tab selected. The profile information is as follows:

HRMS — Employee Dashboard	
Welcome, azeem_11	
My Profile Salary Leaves Security	
Azeem Senior Associate · Finance EMP-1001	
Email azeemworks925@gmail.com	Phone 03189652365
Work Shift Evening Shift · 08:00 AM – 12:00 AM	Monthly Salary PKR 50,000

Employee Dashboard - Salary Tab

The Salary tab shows a list of all salary slips generated for the logged-in employee. The employee can select any slip from the table and click View Selected Slip to open the formatted salary slip window.

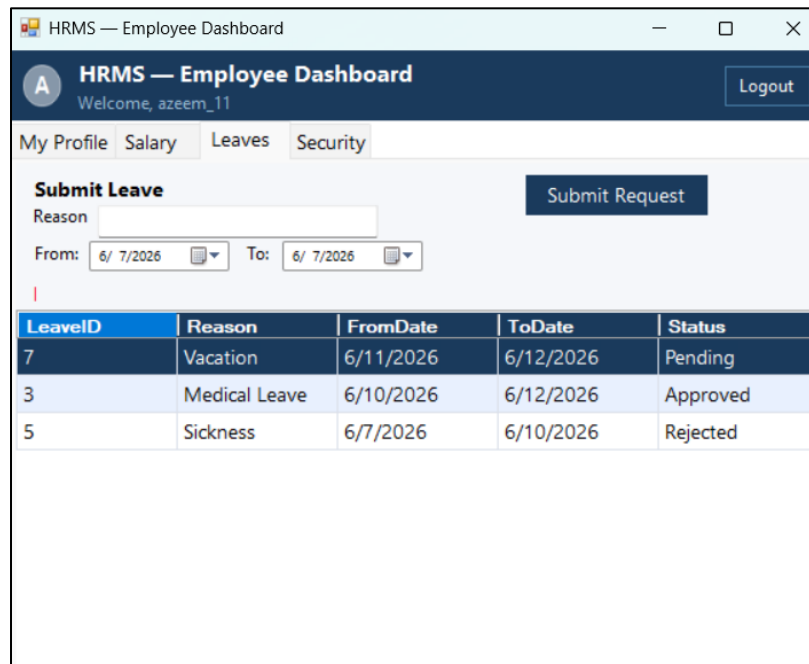


The screenshot shows the HRMS Employee Dashboard with the 'Salary' tab selected. The dashboard header includes a user profile icon, the text 'HRMS — Employee Dashboard', and a 'Logout' button. Below the header, there are tabs for 'My Profile', 'Salary', 'Leaves', and 'Security'. The 'Salary' tab is active, displaying 'My Salary Slips' and a 'View Selected Slip' button. A table lists salary slips with columns: SlipID, Month, GrossSalary, TaxAmount, and NetSalary. The table contains 8 rows of data, with SlipID 15 selected.

SlipID	Month	GrossSalary	TaxAmount	NetSalary
15	June	50000.00	3500.00	46500.00
11	July	50000.00	3500.00	46500.00
9	August	50000.00	3500.00	46500.00
7	February	50000.00	3500.00	46500.00
5	April	50000.00	3500.00	46500.00
4	May	50000.00	3500.00	46500.00
1	March	50000.00	3500.00	46500.00

Employee Dashboard - Leaves Tab

The Leaves tab allows the employee to submit a new leave request by entering a reason and selecting the from and to dates. Below the submission form, all previously submitted requests are displayed along with their current status of Pending, Approved, or Rejected.

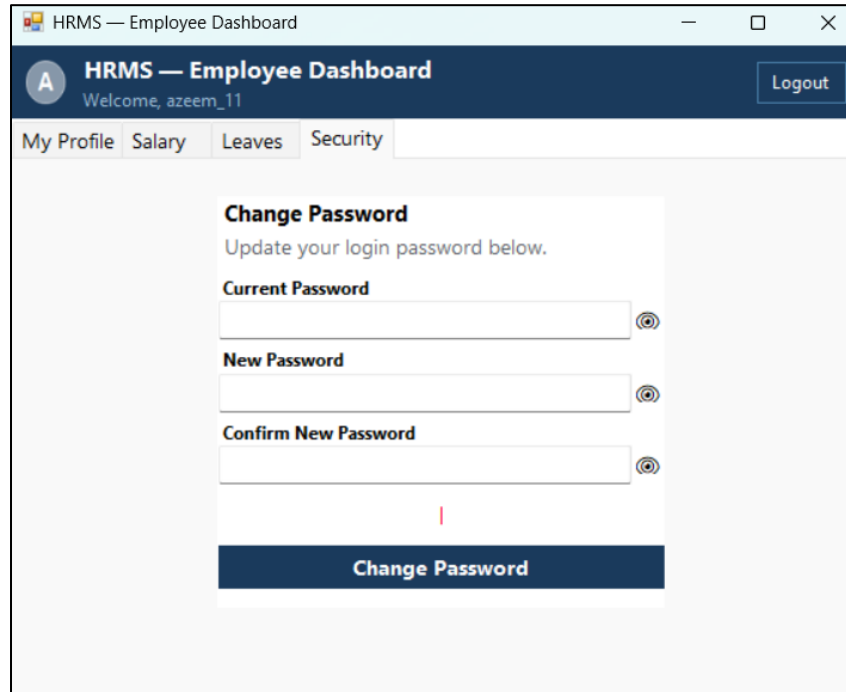


The screenshot shows the HRMS Employee Dashboard with the 'Leaves' tab selected. The dashboard header is the same as the previous screenshot. Below the header, there are tabs for 'My Profile', 'Salary', 'Leaves', and 'Security'. The 'Leaves' tab is active, displaying a 'Submit Leave' form and a 'Submit Request' button. The form includes a 'Reason' text field, a 'From' date field (6/7/2026), and a 'To' date field (6/7/2026). Below the form, a table lists leave requests with columns: LeaveID, Reason, FromDate, ToDate, and Status. The table contains 3 rows of data, with LeaveID 7 selected.

LeaveID	Reason	FromDate	ToDate	Status
7	Vacation	6/11/2026	6/12/2026	Pending
3	Medical Leave	6/10/2026	6/12/2026	Approved
5	Sickness	6/7/2026	6/10/2026	Rejected

Employee Dashboard - Security Tab

The Security tab allows the employee to change their login password. The employee must enter their current password, then the new password twice for confirmation. The system verifies the current password and checks that the new password meets the format requirements before saving.



The screenshot displays the 'HRMS — Employee Dashboard' interface. The top navigation bar includes a user profile icon, the text 'HRMS — Employee Dashboard', 'Welcome, azeem_11', and a 'Logout' button. Below this, a tabbed menu shows 'My Profile', 'Salary', 'Leaves', and 'Security'. The 'Security' tab is active, showing a 'Change Password' form. The form contains three input fields: 'Current Password', 'New Password', and 'Confirm New Password', each with a toggle icon for password visibility. A red vertical line is positioned between the 'New Password' and 'Confirm New Password' fields. At the bottom of the form is a dark blue button labeled 'Change Password'.

CRC Diagram

OOP Concepts Used in Each Class

Interfaces

ISaveable.cs

Interface: ISaveable is an interface that defines a contract. Any class that implements it must provide its own version of Save() and Delete(). The interface itself contains no logic or data, only method signatures.

Abstraction: By defining what operations must exist without defining how they work, ISaveable hides implementation details and only exposes the required behavior to the outside world.

ISearchable.cs

Interface: ISearchable defines a contract requiring any implementing class to provide GetAll() and GetByID() methods. It does not contain any code, only the method declarations.

Abstraction: The interface separates what the data layer must be able to do from how it actually does it. The UI and BL layers only need to know that these methods exist, not how they work internally.

IValidatable.cs

Interface: IValidatable defines a contract requiring any implementing class to provide Validate() and GetValidationError() methods. It is implemented by PersonBL and inherited by all its subclasses.

Abstraction: By declaring these methods in an interface, the system ensures that every class in the person hierarchy can be validated through a common contract without exposing the internal validation logic.

Utility Classes

Validation.cs

Class: Validations is a class that groups all validation logic for the entire application in one place.

Static Members: The class and all its methods are declared as static. This means no object of this class is ever created. Methods are called directly using the class name, for example Validations.PasswordCheck().

Single Responsibility: This class has one job only — to validate input. It does not store data, interact with the database, or handle UI logic. Every method either returns the string "True" if input is valid or returns a specific error message if it is not.

Encapsulation: All validation rules are contained inside this single class. Other classes do not need to know how validation works internally. They simply call the method and receive a result.

Business Logic Layer (BL)

PersonBL.cs

PersonBL is a class that represents a person in the system and holds common personal data shared by all types of users.

Abstraction: PersonBL is declared as abstract. This means it cannot be used to create an object directly. It exists only as a base template for other classes to build upon.

Encapsulation: All fields such as name, email, phone, and address are declared as protected. They cannot be accessed directly from outside the class hierarchy. Getters and setters are provided as public methods to read and write these values in a controlled way.

```
protected string name;  
protected string email;  
protected string phone;  
protected string address;
```

Interface Implementation: PersonBL implements the IValidatable interface. It declares Validate() and GetValidationError() as abstract methods, meaning every subclass must provide its own version of these methods.

```
public abstract class PersonBL : IValidatable
```

Abstract Methods:

```
public abstract bool Validate();  
public abstract string GetValidationError();
```

Inheritance: PersonBL acts as the root base class for the entire person hierarchy. UserBL, EmployeeBL, and AdminBL all inherit from it directly or indirectly.

UserBL.cs

UserBL represents a system user and extends PersonBL by adding login-related information.

Inheritance: UserBL inherits from PersonBL using the colon syntax. It automatically gains all the fields and methods of PersonBL such as name, email, phone, and address without rewriting them.

Encapsulation: The fields username, password, and role are declared as protected. They are only accessible through the provided getter and setter methods.

```
protected string username;  
protected string password;  
protected string role;
```

Constructors: Three constructors are defined. The default constructor initializes all fields to empty strings and sets the role to Employee. The parameterized constructor accepts a username and password directly. The copy constructor creates a new UserBL object by copying all values from an existing one.

```
public UserBL() : base()
```

```
public UserBL(string username, string password) : base()
```

```
public UserBL(UserBL other) : base()
```

Polymorphism (Dynamic): UserBL overrides the abstract methods Validate() and GetValidationError() that were declared in PersonBL. At runtime, if the actual object is a UserBL, its own version of these methods is called.

```
public override bool Validate()
```

```
{  
    return name != "" && username != "" && password != "";  
}
```

```
public override string GetValidationError()
```

```
{  
    if (name == "") return "Name is required.";  
    if (username == "") return "Username is required.";  
    if (password == "") return "Password is required.";  
    return "";  
}
```

EmployeeBL.cs

EmployeeBL represents an employee in the system and is the most complete person class in the hierarchy.

Inheritance: EmployeeBL inherits from UserBL, which itself inherits from PersonBL. This means EmployeeBL automatically has all fields and methods from both parent classes personal information from PersonBL and login information from UserBL.

```
public class EmployeeBL : UserBL
```

Encapsulation: Employment-specific fields such as empID, salary, designation, and deptID are declared as private. They can only be read or written through the provided getter and setter methods.

```
private string empID;  
private double salary;  
private string designation;  
private int deptID;
```

Constructors: Three constructors are provided. The default constructor sets initial values. The parameterized constructor allows creating an employee object with specific values passed in directly. The copy constructor creates a new EmployeeBL object by duplicating all values from an existing one.

```
public EmployeeBL() : base()
public EmployeeBL(string empID, string name, double salary,
string designation, int deptID) : base()
public EmployeeBL(EmployeeBL other) : base(other)
```

Polymorphism (Dynamic): EmployeeBL overrides Validate() and GetValidationError() from PersonBL with its own implementation that checks employment-specific fields like empID, salary, and deptID.

Polymorphism (Static): The CalculateTax() method is overloaded. One version takes no parameters and calculates tax at the default 7% rate. Another version accepts a custom rate as a parameter. Both methods share the same name but have different signatures, which is method overloading.

```
public double CalculateTax() { return salary * 0.07; }
public double CalculateTax(double customRate) { return salary * customRate; }
```

AdminBL.cs

AdminBL represents an administrator in the system and branches off from UserBL separately from EmployeeBL.

Inheritance: AdminBL inherits from UserBL. It reuses all personal and login fields from the parent classes and adds only the adminID field on top.

```
public class AdminBL : UserBL
```

Encapsulation: The adminID field is declared as private and is accessed only through its getter and setter methods.

```
private string adminID;
```

Polymorphism (Dynamic): AdminBL overrides Validate() and GetValidationError() from PersonBL with admin-specific rules that check the adminID field in addition to the base checks.

DepartmentBL.cs

DepartmentBL is a standalone class that represents a department in the organization. It does not extend any other class because a department is not a person and shares no common data with the person hierarchy.

Encapsulation: The fields deptID and deptName are declared as private. They are accessed and modified only through the provided getter and setter methods.

```
private int deptID;
private string deptName;
```

SalarySlipBL.cs

SalarySlipBL represents a salary slip for a specific employee for a specific month.

Encapsulation: All fields including slipID, empID, grossSalary, taxAmount, netSalary, and month are private. They are accessed only through getter and setter methods.

```
private int slipID;  
private string empID;  
private double grossSalary;  
private double taxAmount;  
private double netSalary;  
private string month;
```

Composition: A SalarySlipBL object is directly dependent on an employee. It stores the empID as a reference to the employee it belongs to. If the employee is removed from the system, their salary slips have no meaning on their own.

Encapsulated Business Logic: The SetGrossSalary() method does not simply assign a value. When the gross salary is set, it automatically calculates the tax at 7% and the net salary in the same step. This business rule is hidden inside the setter, which is a clean example of encapsulation of logic.

```
public void SetGrossSalary(double value)  
{  
    grossSalary = value;  
    taxAmount = grossSalary * 0.07;  
    netSalary = grossSalary - taxAmount;  
}
```

LeaveBL.cs

LeaveBL represents a leave request submitted by an employee.

Encapsulation: All fields including leaveID, empID, reason, fromDate, toDate, and status are declared as private. They are only accessible through the provided getters and setters.

```
private int leaveID;  
private string empID;  
private string reason;  
private string status; // Pending, Approved, Rejected  
private string fromDate;
```

```
private string toDate;
```

Composition: A LeaveBL object belongs to a specific employee through the empID field. It has no independent existence without the employee it is linked to.

Data Layer (DL)

UserDL.cs

UserDL is a class that handles all database operations related to the Users table and admin accounts.

Static Members: All methods in UserDL are declared as static. They are called directly using the class name without creating an object, for example UserDL.CheckLogin().

Encapsulation: The connection string is declared as a static field inside the class. All database connection logic is contained within individual methods and is not exposed to the outside.

```
public static string connectionString =
```

```
@ "Server=.\SQLEXPRESS;Database=HRMS_DB;Trusted_Connection=True;" ;
```

Object Usage: UserDL receives AdminBL and UserBL objects as parameters in its methods. It reads data from these objects using their getters to build and execute SQL commands.

```
public static string RegisterAdmin(AdminBL admin)
```

EmployeeDL.cs

EmployeeDL is a class that handles all database operations for the Employees table.

Interface Implementation: EmployeeDL implements both ISaveable and ISearchable. This means it must provide concrete implementations of Save(), Delete(), GetAll(), and GetByID() as required by those interfaces.

```
public class EmployeeDL : ISaveable, ISearchable
```

Encapsulation: The connection string is inherited from UserDL as a static reference. All SQL logic is contained inside individual methods and not exposed externally.

Object Usage: EmployeeDL accepts EmployeeBL objects as parameters. It uses the getters of these objects to extract field values and map them to SQL parameters.

Static Members: GenerateEmpID(), Save(), Search(), Update(), and UpdateDesignation() are all static methods called directly on the class without creating an object.

Transactions: The Save() and Delete() methods use SQL transactions to ensure that if one database step fails, all steps are rolled back together, protecting data integrity.

DepartmentDL.cs

DepartmentDL handles all database operations for the Departments table.

Interface Implementation: DepartmentDL implements ISaveable and ISearchable, providing concrete versions of Save(), Delete(), GetAll(), and GetByID().

Encapsulation: All SQL queries and connection logic are contained inside the class methods and are not visible to the calling code.

Static Members: The Save() method is static and is called directly on the class. The instance methods GetAll(), GetByID(), and Delete() are called on an object of the class.

Business Rule Enforcement: The Delete() method checks whether any employees are still assigned to the department before allowing deletion. This business rule is enforced at the data layer level.

LeaveDL.cs

LeaveDL handles all database operations for the LeaveRequests table.

Interface Implementation: LeaveDL implements ISaveable and ISearchable, providing its own versions of Save(), Delete(), GetAll(), and GetByID().

Encapsulation: All connection and query logic is contained within the class. The calling code only needs to know which method to call and what to pass in.

Object Usage: LeaveDL receives a LeaveBL object in its Save() method and reads the field values from it using getters to construct the SQL insert command.

Static Members: Save() and UpdateStatus() are static methods. GetAll() and GetByID() are instance methods called on an object of the class.

SalaryDL.cs

SalaryDL handles database operations for the SalarySlips table.

Static Members: Both GenerateSlip() and GetSlipsByEmpID() are static methods, called directly on the class without creating an object.

Object Usage: GenerateSlip() accepts a SalarySlipBL object and reads the gross salary, tax amount, net salary, employee ID, and month from it using getters before inserting the record.

Encapsulation: All SQL logic is contained inside the class. The duplicate slip check is handled internally before any insert is attempted, so the calling code does not need to manage this logic itself.

ScheduleDL.cs

ScheduleDL handles all database operations for the WorkSchedules table.

Static Members: All three methods AssignSchedule(), GetScheduleByEmpID(), and GetAll() are static and called directly on the class.

Encapsulation: The logic of deleting an existing schedule before inserting a new one is fully contained inside AssignSchedule(). The calling form simply passes the values and receives a result.

Transactions: AssignSchedule() uses a SQL transaction to wrap both the delete and insert operations. If the insert fails after the delete, the transaction is rolled back so no data is lost.

Forms / UI Layer

LoginForm.cs

LoginForm is a class that represents the login screen of the application and inherits from the WinForms Form class.

Inheritance: LoginForm inherits from Form, which gives it all the properties and behavior of a Windows Form such as the ability to display, close, and contain controls.

Object Creation: Inside the login logic, new objects of AdminDashboard and EmployeeDashboard are created based on the role returned from the database. This is direct object instantiation in response to a runtime condition.

Encapsulation: The login logic, password visibility toggle, and icon drawing are each contained in their own private methods. The form's internal behavior is not exposed outside the class.

Static Member Usage: The form calls static methods from UserDL such as UserDL.IsAdminExists() and UserDL.CheckLogin() without creating a UserDL object.

InnitSetupForm.cs

InitSetupForm is a class that represents the first-time setup screen and inherits from Form.

Inheritance: Inherits from the WinForms Form class.

Object Creation: An AdminBL object is created inside the button click handler. Its setter methods are called to populate it with the values the user typed before passing it to UserDL.RegisterAdmin().

Encapsulation: The header icon drawing logic and button event logic are separated into their own private methods, keeping the code organized and the internal details hidden.

AdminDashboard.cs

AdminDashboard is a class representing the admin's main window with five management tabs.

Inheritance: Inherits from the WinForms Form class.

Object Creation: Objects of EmployeeDL, DepartmentDL, and LeaveDL are created as private fields of the class. These objects are used throughout the form to call data retrieval and manipulation methods.

Encapsulation: Each tab's logic is separated into its own private methods such as LoadEmployees(), LoadDepartments(), and LoadLeaves(). The StyleGrid() helper method is reused across all DataGridViews without repeating code.

Object Passing: When the admin clicks Add Employee or Update Employee, a new form object is created and shown. When Promote/Demote is clicked, a PromoteDemoteForm object is created and passed the selected employee's data.

Static Member Usage: Static methods like `EmployeeDL.Search()`, `EmployeeDL.UpdateDesignation()`, `DepartmentDL.Save()`, and `LeaveDL.UpdateStatus()` are called directly on their classes.

EmployeeDashboard.cs

`EmployeeDashboard` represents the employee's personal dashboard with four tabs.

Inheritance: Inherits from the `WinForms Form` class.

Encapsulation: Profile loading, salary loading, and leave loading are separated into their own private methods. The `GetInitials()` helper method extracts initials from a full name and is reused for drawing the avatar.

Object Creation: A `LeaveBL` object is created inside the leave submission handler and populated with the employee's input before being passed to `LeaveDL.Save()`. A `SalarySlipForm` object is created when the employee views a selected slip.

Static Member Usage: `UserDL.UpdatePassword()` and `SalaryDL.GetSlipsByEmpID()` are called as static methods.

addEmployeeForm.cs

`AddEmployeeForm` is a sub-form that handles the creation of a new employee record.

Inheritance: Inherits from the `WinForms Form` class.

Object Creation: An `EmployeeBL` object is created inside the save button handler. Each setter is called to populate the object with data from the form fields before passing it to `EmployeeDL.Save()`.

Encapsulation: Department loading logic is placed in a separate private method `LoadDepartments()` rather than being written directly inside the `Load` event.

Static Member Usage: `EmployeeDL.GenerateEmpID()` is called statically to assign the next available employee ID to the new object.

UpdateEmployeeForm.cs

`UpdateEmployeeForm` is a sub-form that allows editing an existing employee's details.

Inheritance: Inherits from the `WinForms Form` class.

Object Creation: An `EmployeeBL` object is created and populated with the updated values from the form fields before being passed to `EmployeeDL.Update()`.

Encapsulation: The logic for loading the current employee data into the form fields is placed in a separate private method `LoadEmployeeData()`, keeping the `Load` event clean.

Constructor with Parameter: The form's constructor accepts an `empID` string parameter. This value is stored as a private field and used throughout the form to identify which employee is being updated.

PromoteDemoteForm.cs

`PromoteDemoteForm` is a sub-form dedicated to changing an employee's designation.

Inheritance: Inherits from the WinForms Form class.

Constructor with Parameters: The constructor accepts empID, empName, and currentDesignation as parameters. These are used to populate the display labels and pre-select the current designation in the dropdown at the time the form opens.

Encapsulation: The list of valid designations is defined as a private readonly array inside the class. It cannot be changed from outside the form.

Static Member Usage: EmployeeDL.UpdateDesignation() is called statically when the admin confirms the change.

SalarySlipForm.cs

SalarySlipForm is a display-only sub-form that shows a formatted salary slip.

Inheritance: Inherits from the WinForms Form class.

Constructor with Parameters: The constructor accepts empID, name, department, month, gross salary, tax amount, and net salary as parameters and uses them to populate all the labels on the form immediately when it opens.

Encapsulation: All display logic is contained inside the constructor and the close button handler. There are no public methods or exposed fields. The form simply receives data, displays it, and closes.

Main Driver Program

Program.cs

Program is the entry point class of the application. It contains the Main method which is the first thing that runs when the application starts. The Main method is declared as static because it must be callable by the operating system before any object exists.